EE-222 Data Structures and Algorithms

BS(CE)-2k21

LAB REPORT # 9

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	**5**	**5**	**5**	**15**
Obtained				

_____

Lab Instructor.
Kaynat Rana

Experiment # 9

Implementation of Sorting Algorithms in C++ programming language

Objective:

The objective of this lab is to understand the basic concept of different sorting techniques.

- Selection Sort
- Bubble Sort
- Insertion Sort
- Merge Sort
- Radix Sort

Equipment Used:

Visual studio C++

Procedure:

Sorting Algorithm:

A Sorting Algorithm is used to rearrange a given array or list elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of element in the respective data structure.

For example:

The below list of characters is sorted in increasing order of their ASCII values. That is, the character with lesser ASCII value will be placed first than the character with higher ASCII value.

The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats. Following are some of the examples of sorting in real-life scenarios −

- **Telephone directory:**

Telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.

- **Dictionary:**

Dictionary stores words in an alphabetical order so that searching of any word becomes easy.

Selection Sort:

We want to sort n elements store in an array. It's a simple sorting algorithm which start by looking within the array, the smallest element, and then swap it with the one in the first position, then find the element with the second smallest value and swap it with the element in the second position in the array, and then continue until the (n-1) th element is processed. We will then obtain a sorted array. This method is called selection sort.

Algorithm:

In this algorithm, to search the smallest element of the array to be sorted, we will compare elements between them and will only record the index of the smallest element, and when this element is found, swap it using its index. The pseudo-code of the algorithm is:

```
Selection Sort(int[] t, int n)
begin int i, j, min, temp
for i = 1 to n-1 do
min = i
for j = i+1 to n do
if t[j] < t[min] then
min = j
end if
end for
temp = t[min]
t[min]= t[i]
t[i] = temp
end for
```

Bubble Sort:

Bubble sort is the simplest iterative algorithm operates by comparing each item or element with the item next to it and swapping them if needed. In simple words, it compares the first and second element of the list and swaps it unless they are out of specific order. Similarly, Second and third element are compared and swapped, and this comparing and swapping go on to the end of the list. The number of comparisons in the first iteration are n-1 where n is the number elements in an array. The largest element would be at nth position after the first iteration. And after each iteration, the number of comparisons decreases and at last iteration only one comparison takes place.

Algorithm:

Given an array of n items

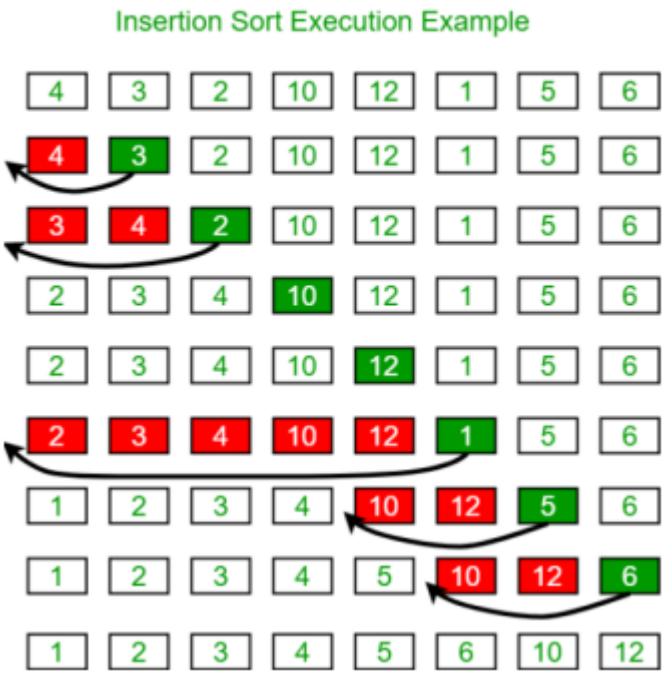
1. Compare pair of adjacent items
2. Swap if the items are out of order
3. Repeat until the end of array - The largest item will be at the last position
4. Reduce n by 1 and go to Step 1

```
Bubble Sort(int [] arr, int n)
int i, j, temp
for i = 0 to n-1 do
for j = 1 to n-i-1 do
if arr[j] > arr[j + 1]
then temp= arr[j]
arr[j]= arr[j+1]
arr[j+1]= temp
end if
end for
end for
```

Insertion Sort:

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element which is to be inserted in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, insertion sort. The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-

list (in the same array). This algorithm is not suitable for large data sets as its average and worst-case complexity are of O (n2), where n is the number of items.



Pseudocode:

```
procedure InsertionSort(A : array of items )
int holePosition
int valueToInsert
for i = 1 to length(A) inclusive do:
/* select value to be inserted */
valueToInsert = A[i]
holePosition = i
/*locate hole position for the element to be inserted */
while holePosition > 0 and A[holePosition-1] > valueToInsert do:
A[holePosition] = A[holePosition-1]
holePosition = holePosition -1
end while
/* insert the number at hole position */
A[holePosition] = valueToInsert
end for
end procedure
```

Merge Sort:

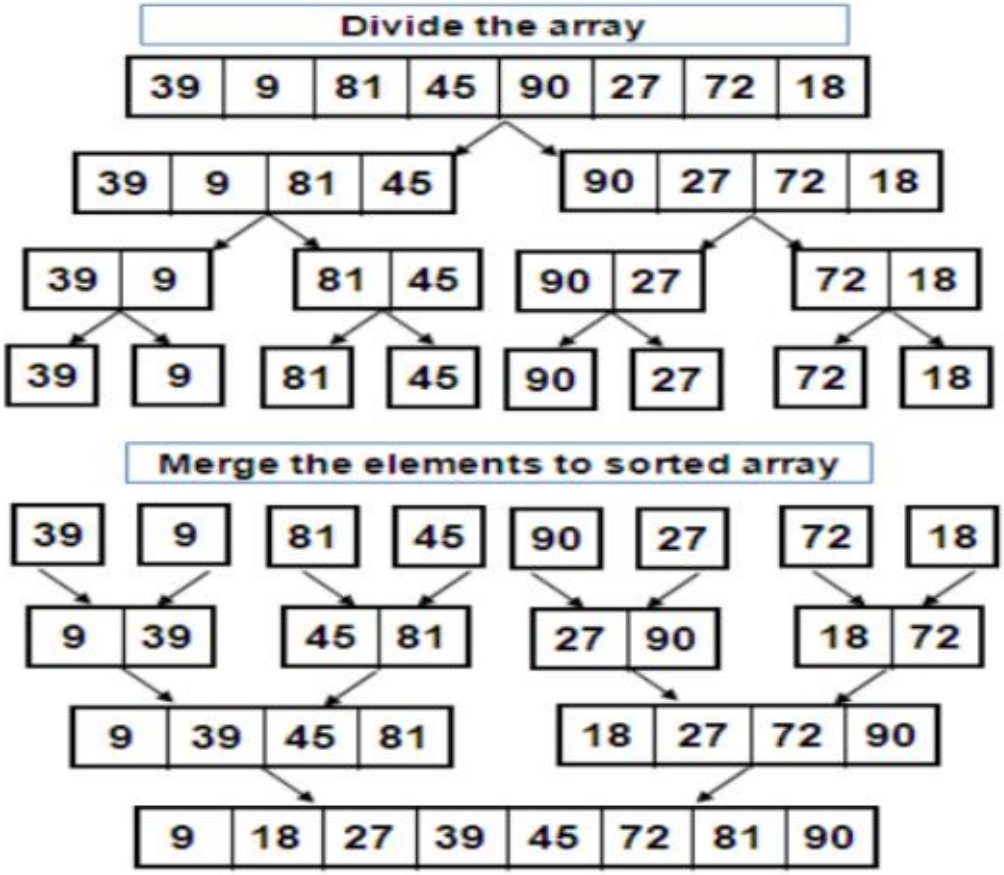
The basic concept of merge sort is dividing the list into two smaller sub-lists of approximately equal size. Recursively repeat this procedure till only one element is left in the sub-list. After this, various sorted sub-lists are merged to form sorted parent list. This process goes on recursively till the original sorted list arrived.

Merge sort is based on the divide-and-conquer paradigm. Since we are dealing with sub-problems, we state each subproblem as sorting a sub-array A[p .. r]. Initially, p = 1 and r = n, but these values change as we recurse through sub-problems. To sort A[p .. r]:

• Divide Step:
If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split A[p .. r] into two sub-arrays A [p .. q] and A[q + 1 .. r], each containing about half of the elements of A[p .. r]. That is, q is the halfway point of A[p .. r].

• Conquer Step:
Conquer by recursively sorting the two sub-arrays A[p .. q] and A[q + 1 .. r].

• Combine Step:
Combine the elements back in A[p .. r] by merging the two sorted sub-arrays A [p…q] and A[q + 1 .. r] into a sorted sequence. To accomplish this step, we will define a procedure MERGE (A, p, q, r).



Algorithm:

```
void merge_sort (int A[ ] , int start , int end )
{
if( start < end )
{
int mid = (start + end ) / 2 ; // defines the current array in 2 parts .
merge_sort (A, start, mid); // sort the 1st part of array.
merge_sort (A, mid+1, end ) ; // sort the 2nd part of array.
merge (A, start , mid , end );
}
}
void merge(int A[ ] , int start, int mid, int end)
{
```

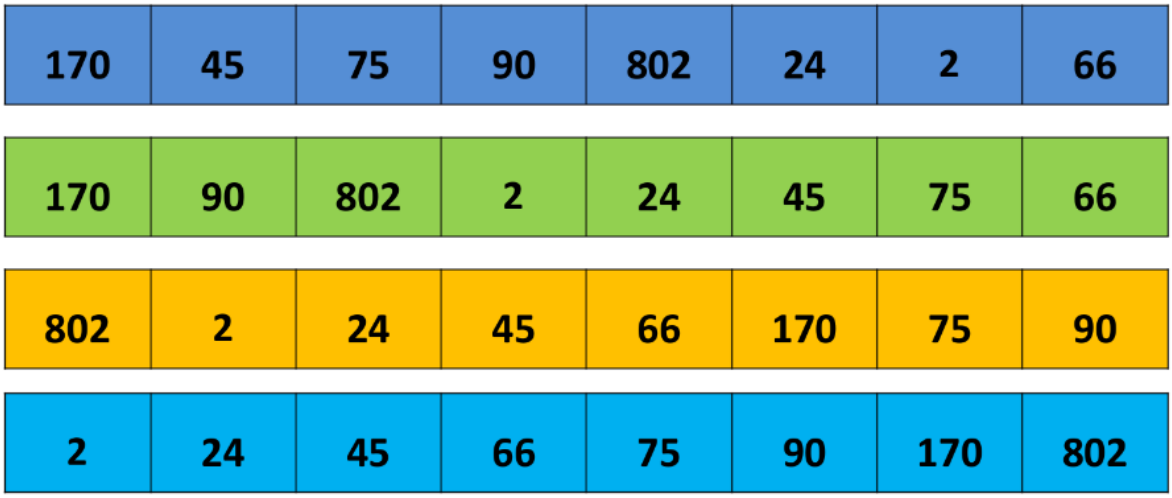
1. Create copies of the subarrays L ← A[start..mid] and M ← A[mid+1..end].
2. Create three pointers i, j and k a. i maintains current index of L, starting at 1 b. j maintains current index of M, starting at 1 c. k maintains current index of A[p..q], starting at p
3. Until we reach the end of either L or M, pick the larger among the elements from L and M and place them in the correct position at A[p..q]
4. When we run out of elements in either L or M, pick up the remaining elements and put in A[p..q]
}

Radix Sort:

Radix sort is an efficient sorting algorithm that sorts data by grouping individual digits of the input numbers. It is often used to sort large numbers, such as integers or floating-point numbers, in linear time.

Algorithm:

- **Step1:** Take the least significant digit of each element
- **Step2:** Sort the list of elements based on that digit
- **Step3:** Repeat the sort with each more significant digit



Tasks

Task 1(i):

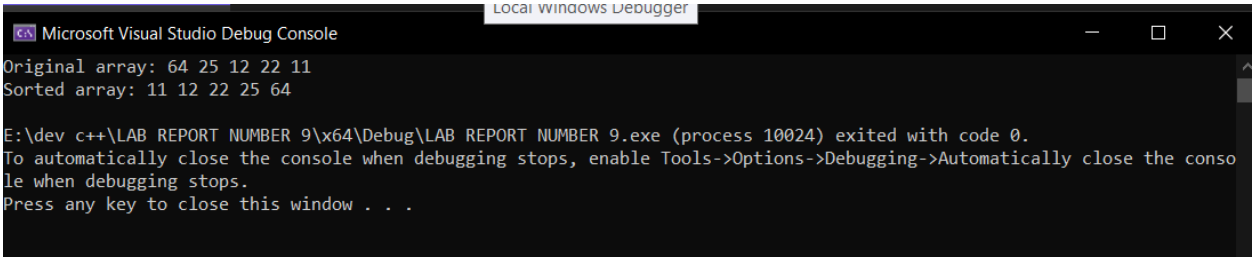
Write C++ codes for all the above given algorithms that we have read in this lab, including Radix Sort.

Selection Sort.

CODE:

```cpp
#include <iostream>
using namespace std;
void selectionSort(int arr[], int size)
{
    for (int i = 0; i < size - 1; ++i)
    {
        int minIndex = i;
        for (int j = i + 1; j < size; ++j)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
}
void printArray(int arr[], int size)
{
    for (int i = 0; i < size; ++i)
    {
        cout << arr[i] << " ";
    }
    cout << std::endl;
}
int main()
{
    int arr[] = { 64, 25, 12, 22, 11 };
    int size = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    printArray(arr, size);
    selectionSort(arr, size);
    cout << "Sorted array: ";
    printArray(arr, size);
    return 0;
}
```

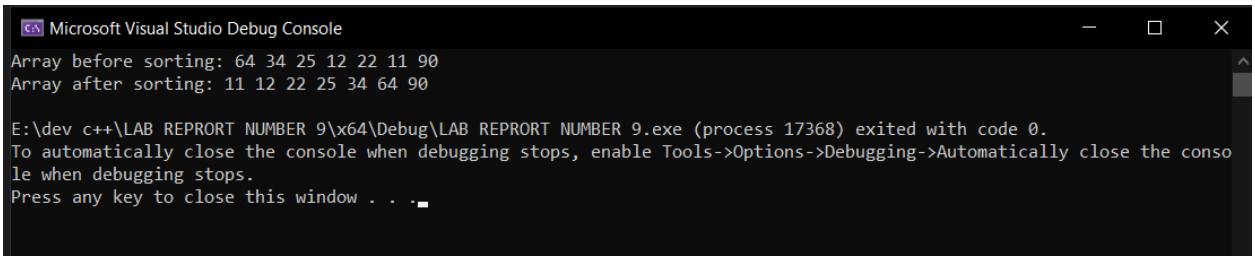
OUTPUT:



Task 1(ii):
Bubble sort
CODE:

```cpp
#include <iostream>
using namespace std;
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
int main()
{
    int arr[] = { 64, 34, 25, 12, 22, 11, 90 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Array before sorting: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    bubbleSort(arr, n);
    cout << "Array after sorting: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT:



Task 1(iii):

Insertion sort

CODE:

```cpp
#include <iostream>
using namespace std;
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
int main()
{
    int arr[] = { 64, 25, 12, 22, 11 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    printArray(arr, n);
    insertionSort(arr, n);
    cout << "Sorted array: ";
    printArray(arr, n);
    return 0;
}
```

OUTPUT:

```
Microsoft Visual Studio Debug Console
Original array: 64 25 12 22 11
Sorted array: 11 12 22 25 64

E:\dev c++\LAB REPORT NUMBER 9(3)\x64\Debug\LAB REPORT NUMBER 9(3).exe (process 16068) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Task 1(iv):

Merge sort

CODE:

```cpp
#include <iostream>
#include <vector>
using namespace std;
void merge(std::vector<int>& arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1);
    vector<int> R(n2);
    for (int i = 0; i < n1; ++i)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; ++j)
        R[j] = arr[mid + 1 + j];
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            ++i;
        }
        else
        {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }
    while (i < n1)
    {
        arr[k] = L[i];
        ++i;
        ++k;
    }
    while (j < n2)
    {
        arr[k] = R[j];
        ++j;
        ++k;
    }
}
void mergeSort(std::vector<int>& arr, int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
void printArray(const std::vector<int>& arr)
{
    int size = arr.size();
    for (int i = 0; i < size; ++i)
        cout << arr[i] << " ";
    cout << std::endl;
}
int main()
{
    vector<int> arr = { 9, 3, 1, 6, 5, 2, 8, 4, 7 };
    int size = arr.size();
    cout << "Original array: ";
    printArray(arr);
    mergeSort(arr, 0, size - 1);
    cout << "Sorted array: ";
    printArray(arr);
    return 0;
}
```

OUTPUT:

```
Microsoft Visual Studio Debug Console
Original array: 9 3 1 6 5 2 8 4 7
Sorted array: 1 2 3 4 5 6 7 8 9

E:\dev c++\LAB REPORT NUMBER 9(4)\x64\Debug\LAB REPORT NUMBER 9(4).exe (process 13380) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

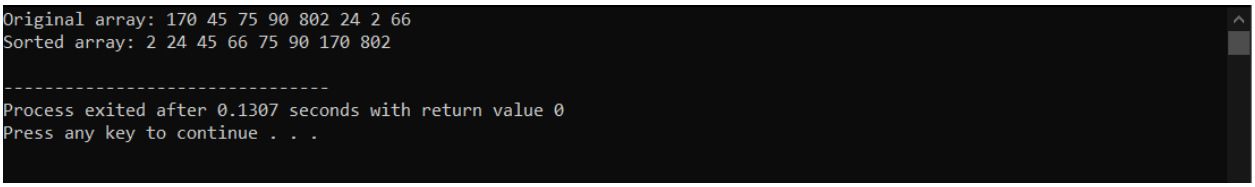
Task 1(v):

Radix Sort

CODE:

```cpp
#include <iostream>
using namespace std;
int getMax(int arr[], int n)
{
    int max = arr[0];
    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}
void countSort(int arr[], int n, int exp)
{
    const int radix = 10;
    int output[n];
    int count[radix] = { 0 };
   for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % radix]++;
    for (int i = 1; i < radix; i++)
        count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--)
    {
        output[count[(arr[i] / exp) % radix] - 1] = arr[i];
        count[(arr[i] / exp) % radix]--;
    }
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}
void radixSort(int arr[], int n)
{
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
int main()
{
    int arr[] = { 170, 45, 75, 90, 802, 24, 2, 66 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    printArray(arr, n);
    radixSort(arr, n);
    cout << "Sorted array: ";
    printArray(arr, n);
    return 0;
}
```

OUTPUT:



Conclusion:

In conclusion, implementing sorting algorithms in C++ programming language can be a challenging task, but it is also a valuable exercise for improving one's understanding of algorithms and data structures. C++ provides a wide range of built-in sorting functions, such as std::sort(), which are efficient and easy to use.